

FoodLoop

Plataforma de generación de recetas con IA

Seguridad, Certificados y Servicios Web

Autenticación, flujo de usuario, servicios
externos e internos

MARTINEZ OTERO JOSUE ATLAI - 230166

RAMIREZ REYES JONATHAN
BALDERMAR - 230410

RODRIGUEZ PEREZ CHRISTIAN PAÚL -
230809

JOSE AGUSTÍN JIMENEZ CASTILLO -
230365



FOODLOOP
ARTE DE COCINAR CON LO QUE H.

Introducción

Este documento complementa el análisis técnico previo de FoodLoop y se enfoca específicamente en el tema de seguridad: cómo se protege la información del usuario, qué tipo de certificados y algoritmos de firma se usan, cuál es el flujo completo por el que un usuario pasa al interactuar con la aplicación, y cómo se integran los servicios web externos e internos que soportan ese flujo.

1. Flujo de interacción del usuario con la aplicación

A continuación se describe el recorrido completo de un usuario dentro de FoodLoop, desde que se registra hasta que genera y consulta recetas. Cada paso está directamente ligado a una ruta de la API y, cuando aplica, a un mecanismo de seguridad.

- 1. Registro de cuenta** — El usuario envía sus datos (nombre, correo, contraseña, edad, aceptación de términos) a POST /api/auth/register. Los datos se validan con express-validator y la contraseña se cifra con bcrypt antes de guardarse; nunca se almacena en texto plano.
- 2. Inicio de sesión** — El usuario envía email y contraseña a POST /api/auth/login. El backend compara el hash con bcrypt y, si es correcto, genera un token JWT firmado con JWT_SECRET, con una vigencia definida por JWT_EXPIRES_IN (7 días). Cada intento (exitoso o fallido) se registra en la tabla login_attempts.
- 3. Autenticación en rutas protegidas** — El token JWT recibido se envía en el header Authorization: Bearer <token> en cada solicitud posterior. El middleware auth valida la firma y vigencia del token y extrae el userId antes de dejar pasar la solicitud al controlador.
- 4. Detección de ingredientes (opcional)** — El usuario sube una foto a POST /api/ingredients/detect (multipart/form-data, campo image). El middleware multer valida tipo (JPG/PNG/WEBP) y tamaño (máx. 5 MB) del archivo antes de enviarlo al servicio de IA de Gemini, que identifica y clasifica los ingredientes visibles.
- 5. Generación de recetas** — Con o sin ingredientes detectados, el usuario solicita POST /api/recipes/generate (ruta protegida con JWT). El servicio de IA arma un prompt con los ingredientes y el perfil del usuario, lo envía a Gemini y guarda el resultado en PostgreSQL (tablas recipes, recipe_ingredients, recipe_steps).
- 6. Consulta y gestión de recetas e ingredientes** — El usuario puede ver sus recetas guardadas (GET /api/recipes/user), marcarlas como favoritas, y administrar su despensa personal (GET/POST/PUT/DELETE /api/ingredients/...), todas rutas protegidas por JWT que garantizan que un usuario solo acceda a su propia información.
- 7. Recuperación de contraseña (si aplica)** — Si el usuario olvida su contraseña, se genera un token de un solo uso guardado en users.reset_password_token con expiración en users.reset_password_expires, y se envía por correo (servicio SMTP) un enlace para restablecerla.

1.1 Diagrama textual del flujo

```
Usuario -> POST /api/auth/register (bcrypt + express-validator) -> POST /api/auth/login
(bcrypt.compare + JWT firmado) -> [token JWT guardado en cliente] -> POST /api/ingredients/detect
(multer + Gemini Vision) -> POST /api/recipes/generate (auth JWT + Gemini) -> GET
/api/recipes/user (auth JWT) -> GET/POST/PUT/DELETE /api/ingredients/* (auth JWT)
```

2. Seguridad implementada

FoodLoop implementa varias capas de seguridad a nivel de aplicación. A continuación se detalla cada mecanismo y el archivo/tabla donde vive.

2.1 Autenticación basada en JWT

Cada usuario autenticado recibe un JSON Web Token firmado con una clave secreta (JWT_SECRET, almacenada en variables de entorno y nunca en el repositorio). El middleware auth se aplica a todas las rutas que requieren sesión, y se encarga de:

- Verificar que el token no haya expirado (JWT_EXPIRES_IN = 7 días).
- Verificar que la firma del token sea válida, es decir, que no haya sido alterado.
- Extraer el userId del payload y adjuntarlo a la solicitud (req.userId) para que los controladores solo devuelvan datos del usuario dueño de la sesión.

2.2 Hashing de contraseñas

Las contraseñas nunca se guardan en texto plano. Se cifran con bcrypt, usando un número de rondas de sal (salt rounds) configurable mediante la variable BCRYPT_ROUNDS. A mayor número de rondas, mayor costo computacional para un atacante que intente romper el hash por fuerza bruta.

2.3 Validación de entradas

La librería express-validator sanitiza y valida los datos recibidos en registro y login (formato de correo, longitud y complejidad de contraseña, coincidencia de confirm_password, aceptación de términos), evitando inyecciones y datos malformados antes de que lleguen a la base de datos.

2.4 Control de acceso por roles

La tabla users incluye un campo role. Las rutas administrativas están protegidas adicionalmente por el middleware isAdmin, que solo permite el acceso a usuarios con el rol correspondiente, separando claramente las funciones de un usuario normal de las de un administrador.

2.5 Protección contra fuerza bruta

Se aplica rate limiting a las rutas sensibles (como login) para limitar el número de solicitudes por unidad de tiempo. Además, cada intento de inicio de sesión —exitoso o fallido— se registra en la tabla login_attempts, lo que permite auditar patrones sospechosos de acceso.

2.6 Recuperación segura de contraseña

El flujo de recuperación usa un token de un solo uso (reset_password_token) con expiración (reset_password_expires). El token se envía únicamente por correo electrónico a través del servicio SMTP configurado, y deja de ser válido después de usarse o de expirar (implementación parcial, requiere SMTP configurado en producción).

2.7 Manejo de archivos subidos

La detección de ingredientes por imagen usa multer con almacenamiento en memoria (memoryStorage), un límite estricto de tamaño (5 MB) y un filtro de tipo de archivo (fileFilter) que solo acepta JPEG, PNG o WEBP, rechazando cualquier otro tipo de archivo antes de que llegue al servicio de IA.

2.8 Gestión de secretos

Todas las claves y credenciales (JWT_SECRET, GEMINI_API_KEY, OPENAI_API_KEY, credenciales de base de datos, credenciales SMTP) se cargan desde variables de entorno (.env) y explícitamente no se suben al repositorio, siguiendo buenas prácticas de manejo de secretos.

2.9 Resumen de mecanismos de seguridad

Mecanismo	Herramienta / Tabla	Propósito
Autenticación	JWT + middleware auth	Verificar identidad del usuario en cada solicitud
Hashing de contraseñas	bcrypt (BCRYPT_ROUNDS)	Evitar almacenamiento de contraseñas en texto plano
Validación de datos	express-validator	Prevenir datos malformados o maliciosos
Control de acceso	campo role + middleware isAdmin	Separar permisos de usuario y administrador
Anti fuerza bruta	rate limiting + login_attempts	Mitigar ataques de adivinación de contraseña
Recuperación de cuenta	reset_password_token/expires	Restablecimiento seguro de contraseña vía correo
Carga de archivos	multer (fileFilter, límite 5 MB)	Evitar archivos maliciosos o excesivamente grandes
Gestión de secretos	Variables de entorno (.env)	Evitar exposición de claves en el repositorio

3. Tipo de certificado y cifrado en tránsito

Además de la seguridad a nivel de aplicación, FoodLoop depende de dos tipos de "certificación" para proteger la información mientras viaja por la red: el tipo/algorithm del token JWT y el certificado TLS/SSL usado en las conexiones externas.

3.1 Certificado y algoritmo del token JWT

Un JWT no es un certificado X.509, pero sí declara un "tipo" y un algoritmo de firma en su cabecera (header), que es lo que garantiza que el token no pueda ser falsificado sin conocer el JWT_SECRET:

```
Header: {  "alg": "HS256",    // algoritmo de firma HMAC con SHA-256  "typ": "JWT"        // tipo de token } Payload: { "userId": 1, "iat": ..., "exp": ... } Firma: HMACSHA256(base64(header) + "." + base64(payload), JWT_SECRET)
```

El algoritmo HS256 (HMAC-SHA256) es simétrico: la misma clave (JWT_SECRET) se usa para firmar y para verificar el token. Esto significa que el secreto debe protegerse con el mismo cuidado que una contraseña maestra, ya que quien lo conozca podría emitir tokens válidos.

3.2 Certificado TLS/SSL en las comunicaciones

Para proteger los datos en tránsito (contraseñas, tokens, imágenes, respuestas de IA), FoodLoop se apoya en certificados TLS/SSL en distintos puntos de la arquitectura:

Conexión	Tipo de cifrado	Detalle
Cliente ↔ API FoodLoop	HTTPS (TLS) — recomendado en producción	En desarrollo corre en HTTP simple (localhost:3001); en producción debe exponerse detrás de un certificado TLS (por ejemplo, emitido por Let's Encrypt o el proveedor de hosting)
API ↔ Servidor SMTP	STARTTLS (puerto 587) o SSL implícito (puerto 465)	El transportador de Nodemailer activa secure: true automáticamente si SMTP_PORT es 465; en 587 usa STARTTLS
API ↔ Google Gemini / OpenAI	HTTPS (TLS) gestionado por el SDK	Las librerías oficiales (@google/generative-ai, openai) fuerzan HTTPS en todas sus solicitudes
API ↔ PostgreSQL	Depende del proveedor	En hosting administrado (por ejemplo, RDS o Supabase) se recomienda forzar sslmode=require

En resumen: el JWT usa un esquema de firma simétrica (HS256) para garantizar la integridad del token en sí, mientras que los certificados TLS/SSL protegen la confidencialidad de los datos mientras viajan entre el cliente, la API de FoodLoop y los proveedores externos (Gemini, OpenAI, SMTP).

4. Servicios web externos

FoodLoop depende de proveedores externos para funciones que no tiene sentido reconstruir internamente: generación de contenido con IA y envío de correo.

Servicio externo	Variable de entorno	Uso en FoodLoop
Google Gemini	GEMINI_API_KEY	Generación de recetas y detección de ingredientes por imagen (modelo gemini-2.5-flash)
OpenAI	OPENAI_API_KEY	Alternativa / respaldo al servicio de generación de recetas mediante IA
Servidor SMTP	SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASS	Envío de correos de verificación de cuenta y recuperación de contraseña

Estos servicios se comunican siempre mediante HTTPS/TLS y sus credenciales se gestionan exclusivamente por variables de entorno, nunca embebidas en el código fuente.

5. Servicio web interno — routes/ingredients.js

Este archivo define el router de Express para todo lo relacionado con ingredientes: detección por imagen, búsqueda, y gestión de la despensa personal del usuario (user_ingredients). Es un buen ejemplo de cómo conviven rutas públicas y protegidas dentro del mismo módulo.

5.1 Configuración de subida de archivos (multer)

El router configura multer con almacenamiento en memoria (no se guarda el archivo en disco), un límite de 5 MB por imagen, y un fileFilter que solo acepta los tipos image/jpeg, image/png e image/webp, rechazando cualquier otro formato con un error explícito.

5.2 Rutas públicas

Método	Ruta	Función	Autenticación
POST	/api/ingredients/detect	Detecta ingredientes desde una imagen subida (campo image)	No requiere
GET	/api/ingredients/search	Busca ingredientes por nombre o categoría	No requiere

Estas rutas son públicas de forma intencional: permiten que un visitante pruebe la detección de ingredientes o busque en el catálogo sin necesidad de crear una cuenta, como "demo" de la funcionalidad principal.

5.3 Rutas protegidas (requieren JWT)

Método	Ruta	Función
GET	/api/ingredients/user	Obtiene los ingredientes guardados en la despensa del usuario autenticado
GET	/api/ingredients/stats	Estadísticas de la despensa del usuario (nuevo)
GET	/api/ingredients/expiring	Ingredientes próximos a caducar (nueva ruta)
POST	/api/ingredients	Añade un ingrediente manualmente a la despensa (nuevo)
PUT	/api/ingredients/:id	Actualiza un ingrediente existente (nuevo)
DELETE	/api/ingredients/:id	Elimina un ingrediente de la despensa (nuevo)

Todas estas rutas usan el middleware auth antes de llegar al controlador, lo que garantiza que un usuario solo pueda ver, crear, modificar o eliminar los ingredientes que le pertenecen a él, nunca los de otro usuario.

5.4 Documentación con Swagger

Cada ruta está documentada con comentarios JSDoc que Swagger interpreta para generar documentación interactiva en /api-docs. Las rutas protegidas declaran explícitamente security: bearerAuth: [], lo que hace que la propia documentación indique qué endpoints requieren token y cuáles no.

5.5 Resumen técnico del servicio interno

Aspecto	Detalle
Framework	Express (Router)
Carga de archivos	multer, memoryStorage, límite 5 MB, filtro JPG/PNG/WEBP
Autenticación	middleware auth (JWT) en rutas protegidas
Documentación	Swagger / JSDoc, expuesta en /api-docs
Dependencia externa	Google Gemini (vía ingredientController) para /detect

6. Conclusión

FoodLoop combina seguridad a nivel de aplicación (JWT, bcrypt, validación, rate limiting, control de acceso por roles) con cifrado en tránsito (TLS/SSL) hacia sus proveedores externos (Gemini, OpenAI, SMTP). El flujo de usuario —registro, login, detección de ingredientes, generación y gestión de recetas— está diseñado para que cada paso sensible pase por una capa de verificación, ya sea autenticación por token o validación de datos. El servicio interno de ingredientes ejemplifica este equilibrio: expone rutas públicas para facilitar la adopción, y protege con JWT todo lo que toca datos personales del usuario.

Como roadmap pendiente, destaca migrar el flujo de recuperación de contraseña a un SMTP verificado en producción, implementar refresh tokens para sesiones más seguras, y mover la carga de imágenes a un almacenamiento externo (Cloudinary/S3) en lugar de mantenerlas en memoria o base64.